TRI CLOUD ESSENTIALS — AWS

- **Topic:** AWS I — Identity and Storage
- **Date:** 29th October 2026

1. AWS IAM: Users, Roles, Policies, and Trust Relationships

AWS (Amazon Web Services) is Amazon's cloud platform, a large collection of services for computing, storage, databases, and security that run in data centers around the world. Everything in AWS lives inside an AWS account, which is the container for your resources and also the boundary for billing and access.

In simple words. IAM (Identity and Access Management) is the service that decides who can do what on which AWS resource. It is global, so it is not tied to one Region, and there is no charge for using it.

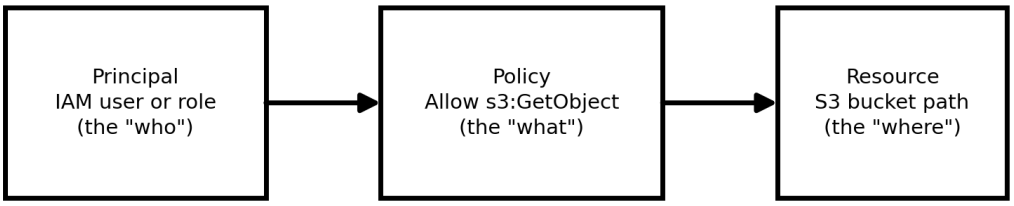
Piece	Simple meaning
Root user	The identity created together with the account, using the sign-up email. It has unlimited power, so protect it with MFA, keep its password safe, and do not use it for daily work.
IAM user	A long-term identity for one person or one application, with a name and credentials such as a console password or access keys. For people, AWS now prefers signing in through IAM Identity Center, which gives temporary credentials.
IAM group	A set of users that share the same permissions. A policy attached to the group applies to every user in it. A group cannot contain another group.
IAM role	An identity with permissions but with no password and no access keys of its own. Someone or something assumes the role and receives temporary credentials.
Policy	A JSON document that lists which actions are allowed or denied, on which resources, and under which conditions.
Access keys	A long-lived pair of an access key ID and a secret access key, used by code to call AWS. They are easy to leak, so prefer roles.

Policies. Permissions are never given directly. They are written in a policy and the policy is attached to an identity or to a resource.

Policy type	Simple meaning

Identity-based policy	Attached to a user, group, or role. It says what that identity may do.
Resource-based policy	Attached to a resource, such as an S3 bucket policy. It names the principal who may use the resource, so it contains a Principal element.
Permissions boundary	A limit set on a user or role. The identity can do only what both its own policies and the boundary allow.
Service control policy (SCP)	A guardrail set in AWS Organizations for whole accounts. It limits what the accounts can do, but it grants nothing by itself.
Element	**What it says**
Version	The policy language version. It is always "2012-10-17".
Statement	The list of rules. Each rule is one block inside the policy.
Effect	Either Allow or Deny.
Action	The API call, written as service:Operation, such as s3:GetObject.
Resource	The thing the action is on, written as an ARN, such as arn:aws:s3:::tricloud-datalake-dev-001/sales/*.
Principal	Who the rule is for. It appears only in resource-based policies and trust policies.
Condition	An extra test, such as the request coming from a given IP address or being signed in with MFA.

An IAM request: who, what, on which resource



Everything is denied by default, and an explicit Deny always beats an Allow

Diagram 1: An identity, a policy, and a resource together decide whether a request is allowed.

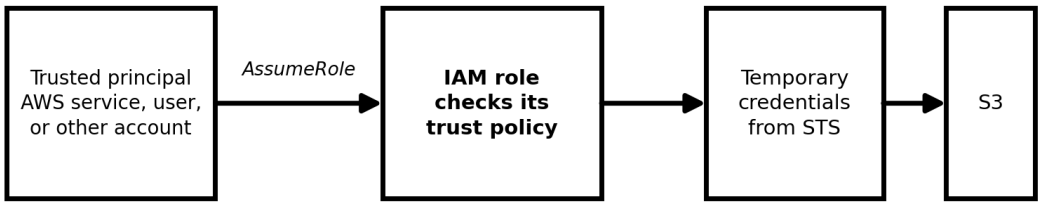
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSalesFiles",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::tricloud-datalake-dev-001/sales/*"
    }
  ]
}
```

How AWS decides. Every request starts as denied. An Allow in a matching policy opens it, and an explicit Deny anywhere closes it again, whatever else says Allow. Inside one account, an Allow in either the identity-based policy or the resource-based policy is enough. Between two accounts, both sides must allow.

Policy kind	Simple meaning
AWS managed policy	Written and updated by AWS, such as AmazonS3ReadOnlyAccess. It is quick to use but often broader than needed.
Customer managed policy	Written by you and reusable. It can be attached to many identities and has versions you can roll back.
Inline policy	Written directly inside one user, group, or role and deleted with it. It is used for a one-off rule.

Roles and trust relationships. A role has two separate policies. The trust policy (also called the trust relationship) says who is allowed to assume the role. The permissions policy says what the role can do after it is assumed. When someone assumes the role, the AWS Security Token Service (STS) hands out temporary credentials: an access key, a secret key, and a session token. They expire, one hour by default, so there is nothing permanent to leak.

How a role is assumed



The trust policy says who may assume the role. The permissions policy says what the role can do.

Diagram 2: The trust policy lets a principal assume the role and receive temporary credentials.

Who can be trusted	**Example**
An AWS service	ec2.amazonaws.com, lambda.amazonaws.com, or glue.amazonaws.com. This is how a service gets its own permissions.
A user or role in the same account	A developer's role that is allowed to switch into an admin role for a task.
Another AWS account	A partner or a separate account for production. The Principal names that account's ID.
An outside identity	A user from a company directory or a CI/CD tool, signed in through SAML or OpenID Connect (OIDC) federation.
Step	**What to do**
1. Open Roles	In the IAM console choose Roles, then Create role.
2. Choose the trusted entity	Pick AWS service, for example Glue, EC2, or Lambda. This writes the trust policy for you.
3. Attach permissions	Attach a managed policy, or a customer managed policy that you wrote for this job.
4. Name and create	Use a clear name such as tricloud-etl-role and add tags.
5. Check it	Use the policy simulator or the Last accessed tab to confirm what the role really uses.

2. The Principle of Least Privilege

In simple words. Least privilege means giving an identity only the permissions it needs for its job, on only the resources it needs, and only for as long as it needs them. If something goes wrong, such as a leaked key or a mistake in a script, the damage stays small.

Dimension	What to limit
Actions	Allow the exact API calls, such as s3:GetObject, and not s3:* or *.
Resources	Name the exact ARN or prefix, such as the sales/ prefix of one bucket, and not every bucket.
Conditions	Add tests such as MFA required, a certain source IP range, or an allowed Region.
Time	Use roles with temporary credentials, which expire, and not access keys that live for years.

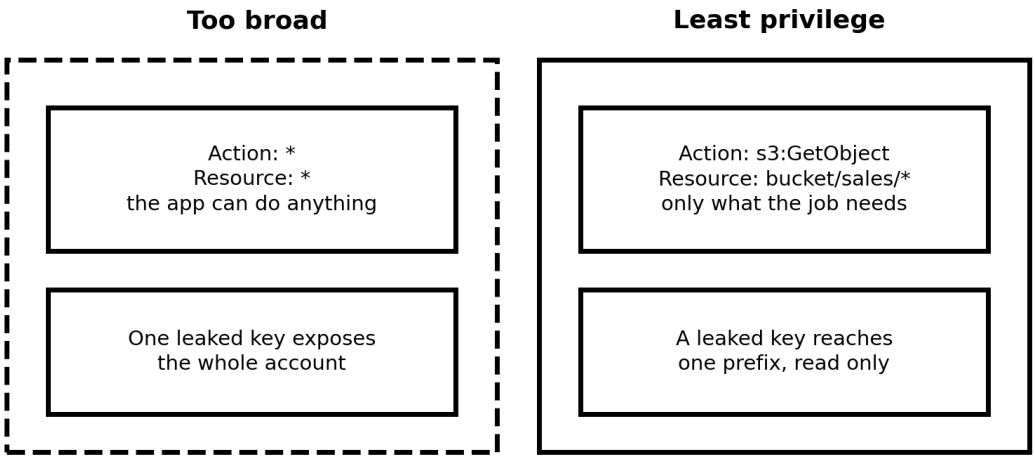


Diagram 3: A broad policy and a least-privilege policy, and what a leaked key can reach in each.

Note. The first rule works on the bucket itself and the second works on the objects inside it. This is why two different ARNs appear. The condition keeps the listing inside the sales/ prefix.

Step	What to do
1. Start small	Begin with no permissions, or with a narrow managed policy, and add only what a real task needs.
2. Write a scoped policy	Replace the broad managed policy with a customer managed policy that names exact actions and ARNs.

3. Let AWS suggest	IAM Access Analyzer can build a policy from the activity recorded in CloudTrail, so the policy matches what was actually used.
4. Remove what is unused	Check the last accessed information of each user and role, and delete the permissions and the access keys that are not used.
5. Add guardrails	Use permissions boundaries and SCPs so that no one can grant more than the organization allows.
6. Review often	Permissions grow over time, so review them on a schedule, for example every quarter.